DecisionAI: Complete Engineering Journey

From Inception (Phase 0) to Thought-Partner Meeting Intelligence

Stack: FastAPI + MV3 + ChromaDB **Timeline:** Phase 0 through Phase 8 **Status:** 61/61 Tests Passing (Production)

1. Executive Summary & Vision

DecisionAI was built to replace clumsy, invasive meeting bots with native browser capture and active thought-partner intelligence. Traditional bots require calendar invitations, appear as unwanted accounts in call recordings, and drop speech context during bilingual English, Hindi, and Hinglish discussions. DecisionAI solves these limitations with direct dual-channel audio capture, sub-second 16 kHz AudioWorklet processing, pluggable multilingual speech recognition, and hybrid RAG search.

- **Bot-Free Capture:** Records tab and mic audio directly via a Manifest V3 extension.
  - **Signal Precision:** Real-time 16 kHz mono Int16 PCM conversion via dedicated AudioWorklets.
  - **Linguistic Inclusivity:** Accurate transcription across English, Hindi, and Hinglish code-switching.
  - **Thought-Partner AI:** Synthesizes decisions and provides constructive technical recommendations rather than serving as a passive archive.

2. Phase 0: Project Skeleton & Configuration Management

Phase 0 established the project scaffolding, dependency management, and centralized settings. The backend utilizes FastAPI for asynchronous concurrency, SQLite with aiosqlite for zero-config persistence, and Pydantic Settings as the single source of truth.

- **Centralized Config:** Implemented in backend/config.py via pydantic-settings loading .env parameters.
  - **FastAPI Application:** Root routing, CORS middleware for extension communication, and async startup lifecycles.
  - **Design Tokens:** Established a zero-framework vanilla frontend standard with strict WCAG AA contrast rules.

> **PHASE 0 CHALLENGE & SOLUTION:**
> Inconsistent environment variable overrides caused test runs to point to unintended database files. Solution: Configured pydantic-settings with explicit .env precedence and standardized test-isolated SQLite in-memory fixtures.

3. Phase 1: Relational Schema & Persistence Models

Phase 1 delivered SQLAlchemy 2.0 declarative async data models representing the meeting lifecycle, speaker dialogue, extracted decisions, and user auditing.

- **Meeting Model:** Tracks lifecycle transitions (CREATED to RECORDING to PROCESSING to READY to FAILED) with duration metrics.
  - **TranscriptSegment Model:** Stores speaker turns (You vs Participant), start/end millisecond offsets, raw text, and language codes.
  - **Decision Model:** Stores extracted decisions, action items, assignees, deadlines, and status (accepted, rejected, pending).
  - **AuditLog Model:** Records user actions, IP addresses, and timestamps for governance.

PHASE 1 CHALLENGE & SOLUTION:

SQLAlchemy 2.0 async sessions threw greenlet errors when serializing lazy-loaded relationship attributes. Solution: Enforced selectinload on all relationship queries and decoupled API endpoints from database models using strict Pydantic response schemas.

4. Phase 2: Audio File Upload & Storage Management

Phase 2 introduced manual file ingestion via POST /api/meetings/upload to support legacy meeting audio, file validation, and offline testing workflows.

- **Payload Verification:** Magic-byte binary verification for WAV, MP3, M4A, and WebM audio formats.
  - **Storage Safeguards:** Strict 500 MB upload ceiling and duration extraction via soundfile and wave.
  - **Deterministic Organization:** Organized audio artifacts under user-isolated directories.

PHASE 2 CHALLENGE & SOLUTION:

Operating systems transmitted non-standard MIME headers (e.g. audio/x-wav vs audio/wav). Solution: Paired MIME header parsing with binary magic-byte inspection, guaranteeing robust upload validation across platforms.

5. Phase 3: Manifest V3 Extension & Dual-Channel Live Capture

Phase 3 accomplished live audio capture directly from Google Meet without meeting bots, overcoming stringent Manifest V3 sandbox restrictions.

- **chrome.tabCapture:** Service worker obtains restricted streamId without elevated intrusive permissions.
  - **Offscreen Document:** Hosts AudioContext and navigator.mediaDevices.getUserMedia for participant tab and user mic.
  - **Dual AudioWorklets:** PCMRecorderProcessor resamples audio streams to 16 kHz mono signed 16-bit PCM.
  - **Live Binary Streaming:** Streams 1-second chunks over WebSocket framed with ASCII headers (TAB: and MIC:).

PHASE 3 MAJOR PROBLEM & SOLUTION: TAB MUTING:

Capturing tab audio via chrome.tabCapture automatically muted the call in Chrome! The user could record participants but could not hear them. Solution: In offscreen.js, the participant stream was split and piped into audioCtx.destination. This maintained crystal-clear audio to the user's headphones while concurrently streaming samples to the recording worklet.

PHASE 3 MAJOR PROBLEM & SOLUTION: WEBSOCKET AUTH & 0.0s DURATION:

Unauthenticated WebSocket handshakes failed with HTTP 403, causing the recording to disconnect and persist 0.0 seconds of audio. Solution: Built the /auth/extension-token endpoint. The popup authenticates the user, obtains a secure stream token, and passes it to the offscreen worker to inject into the WebSocket handshake.

6. Phase 4: Pluggable Multi-Provider STT & Normalization

Phase 4 delivered a multi-provider transcription pipeline capable of handling regional Indian dialects, Devanagari Hindi, English, and romanized Hinglish code-switching.

- **Provider Architecture:** Pluggable interfaces for faster-whisper (local int8), Google Gemini, Sarvam AI (saaras:v4), and Deepgram.
  - **Window Chunking & Channel Merge:** 30-second windows with 0.5s overlap; concurrent Tab and Mic processing chronologically merged by start_ms.
  - **Hindi Normalization:** Unicode NFC normalization, nukta/danda cleanup, and phonetic Hinglish canonical mapping.

PHASE 4 MAJOR PROBLEM & SOLUTION: SILENCE HALLUCINATIONS:

When users were alone on calls, the participant Tab channel recorded pure digital silence (RMS = 0.00). Gemini STT previously hallucinated 1878 historical text (Vernacular Press Act) on silence! Solution: Implemented is_silent_wav pre-flight energy gating. The pipeline computes Root-Mean-Square (RMS) amplitude on raw audio buffers; if pure digital silence is detected, transcription is immediately bypassed with zero hallucination.

7. Phase 5: Authentication, User Scoping & Enterprise Governance

Phase 5 established security and privacy compliance across all application layers.

- **Google OAuth 2.0:** Real OAuth authorization with httpOnly=True, samesite='lax' session cookies.
  - **Tenant Isolation:** Strict user_id enforcement on all SQL queries, meeting viewers, and ChromaDB vector filters.
  - **In-Memory Rate Limiting:** Sliding-window limiter enforcing a 60 request/min cap without external Redis dependencies.
  - **Privacy Wipeout:** Single-action endpoint (/api/user/delete-all) that permanently purges all audio, transcripts, and embeddings.

PHASE 5 CHALLENGE & SOLUTION:

CSRF protection middleware blocked legitimate POST requests from the Chrome extension. Solution: Configured selective CSRF exemptions for Bearer extension token authenticated endpoints while maintaining cookie protection for browser sessions.

8. Phase 6: Post-Processing Intelligence & Structured Decisions

Phase 6 transformed unstructured speech transcripts into executive summaries and structured decision records.

- **Glossary Cleaning:** Technical domain terms (DecisionAI, ChromaDB, AudioWorklet) corrected in transcripts while preserving raw text.
  - **Executive Summary:** Concise overview of meeting topics, participant positions, and conclusions.
  - **Decision Extraction:** Structured classification of commitments into accepted, rejected, and pending states with assignees and timestamps.

PHASE 6 CHALLENGE & SOLUTION:

The extension popup previously froze at 86% processing and never redirected to the meeting viewer. Solution: Identified that popup.js was polling the generic meetings list instead of the specific meeting ID. Updated polling to query /api/meetings/{id} directly and automatically redirect when status reached 'ready'.

9. Phase 7: Responsive Vanilla Frontend & Bento Grid Dashboard

Phase 7 constructed a zero-framework vanilla web frontend (/web) designed for high performance and accessibility.

- **Bento Grid Dashboard:** Modular CSS Grid layout organizing recent meetings, quick metrics, decisions, and chat entry.
  - **WCAG AA Compliance:** Contrast ratios strictly exceed 5.0:1 across dark and light palettes.
  - **Dual Audio Player:** Interactive waveform seeking with automatic transcript dialogue highlighting.
  - **Copy Refactor:** Replaced internal engineering jargon with intuitive executive copy across web and extension.

10. Phase 8: Hybrid RAG, Thought-Partner Advisory AI & Resilience

Phase 8 delivered the conversational intelligence engine, transforming DecisionAI from a storage box into an active, intelligent thought partner.

- **Speaker-Turn Chunking:** Chunks of 300 to 500 tokens respecting conversational boundaries.
  - **ChromaDB & BM25 Hybrid Search:** Fused using Reciprocal Rank Fusion (RRF with k=60) for balanced dense and

lexical retrieval.
- **Grounded Citations:** Cites exact meeting titles, dates, and MM:SS timestamps with interactive audio seeking.

PHASE 8 MAJOR PROBLEM & SOLUTION: THOUGHT-PARTNER ADVISORY MODE:

When users asked consultative or value-add questions (e.g. 'do u think we can add anything else valuable to it'), the assistant refused with: 'No relevant meeting discussions found...'. The AI was acting like a rigid keyword search tool rather than an intelligent partner. Solution: Introduced is_advisory_query classification, separating factual recall queries (grounded strictly in meeting records) from advisory queries (which synthesize meeting context and apply AI intellect and reasoning to deliver concrete architectural recommendations). Updated offline fallbacks to synthesize domain-grounded suggestions rather than quitting.

PHASE 8 MAJOR PROBLEM & SOLUTION: MULTI-MODEL QUOTA RESILIENCE:

Gemini free-tier experienced intermittent HTTP 429 quota limits and HTTP 503 high-demand errors, causing RAG queries to fail unexpectedly. Solution: Implemented tiered candidate model cascading across gemini-3.5-flash-lite, gemini-flash-lite-latest, gemini-3.1-flash-lite, and gemini-3.5-flash with exponential backoff retries and dynamic temperature adjustment (0.4 for advisory, 0.2 for factual recall).

11. Complete Journey Milestone Matrix

Phase	Milestone / Scope	Key Problem Encountered	Technical Solution
Phase 0	Project Skeleton & Config	Inconsistent config paths across OS environments	Pydantic Settings with single source of truth
Phase 1	Database Schema & CRUD	SQLAlchemy async greenlet spawning on lazy relations	Explicit selectinload and dedicated Pydantic DTOs
Phase 2	Audio Upload & Storage	MIME type variations across operating systems	Magic-byte binary inspection and size validation
Phase 3	Chrome MV3 Live Capture	Tab capture automatically muted browser call audio	Split stream to audioCtx.destination in offscreen
Phase 4	Pluggable STT Pipeline	Gemini STT hallucinated history text on silent calls	Root-Mean-Square (RMS) pre-flight silence gating
Phase 5	Auth & Data Governance	CSRF middleware blocked Chrome extension requests	Exempted Bearer token authenticated extension routes
Phase 6	Intelligence Extraction	Popup UI froze at 86% during meeting processing	Targeted meeting ID polling and automated browser redirect
Phase 7	Vanilla Web Dashboard	Technical jargon confused executive end users	Comprehensive copy refactor across UI and extension
Phase 8	Thought-Partner RAG	Assistant refused consultative queries as 'not found'	Added is_advisory_query, thought-partner prompt, 429 backoff